Neural Networks and Deep Learning

Language Models and Advanced NN Concepts

CS224N Lecture Notes

Contents

Lecture Overview

- Advanced neural network concepts: regularization, dropout, optimization
- Introduction to language models
- N-gram language models (classical approach)
- Transition to neural language models
- Recurrent neural networks preview

1 Modern Neural Networks: Key Concepts

1.1 Historical Context: The Deep Learning Revolution

The Problem (Pre-2006)

"Empirically, deep networks were generally found to be not better and often worse than neural networks with one or two hidden layers." — Bengio et al., 2006
For approximately 15 years (1990s-2000s), despite having backpropagation and the theory of deep networks, practitioners could not successfully train networks with more than 1-2 hidden layers.

The Breakthrough: Starting around 2006-2010, several key innovations enabled training of deep networks:

1. Better regularization techniques (especially Dropout)
2. Proper parameter initialization
3. Advanced optimizers (Adam, etc.)
4. Vectorization and GPU acceleration

1.2 Regularization: Classical vs. Modern View

1.2.1 L2 Regularization

The classical regularization approach adds a penalty term to the loss function:

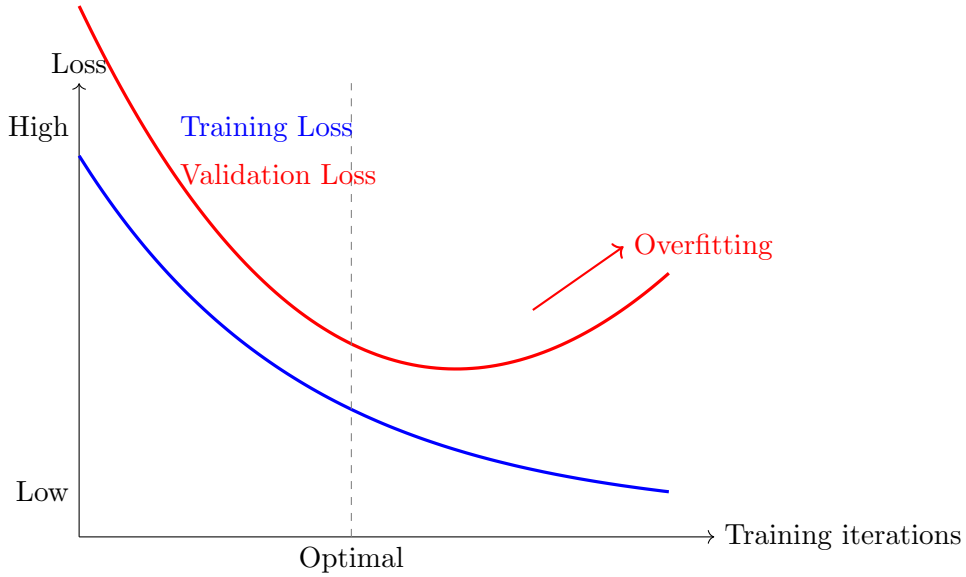
$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i \theta_i^2 \tag{1}$$

where:

- $\mathcal{L}_{\text{data}}$ is the loss on training data
- λ is the regularization strength
- $\sum_i \theta_i^2$ penalizes large parameter values

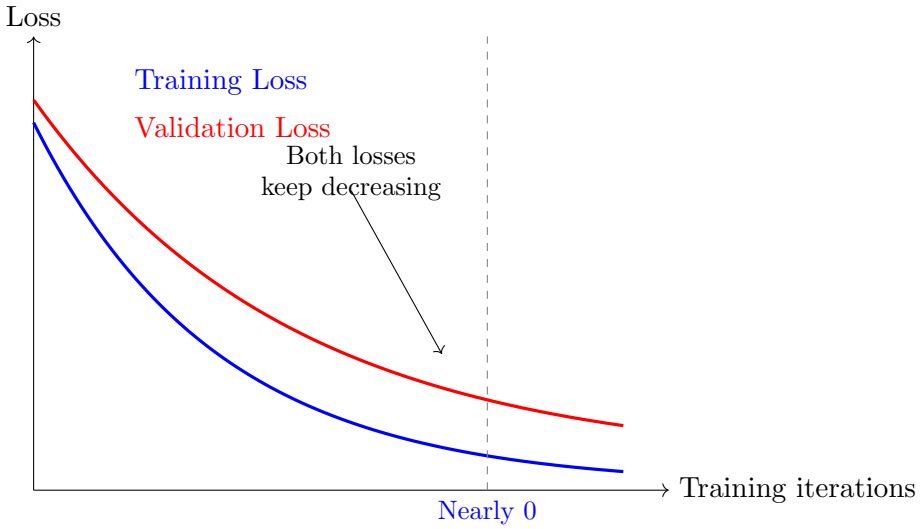
Intuition: Prefer models with smaller parameter weights that still explain the data well.

1.2.2 Classical View of Overfitting



Classical View: Stop training when validation loss starts increasing.

1.2.3 Modern View: No Overfitting Paradox



Modern View: In large networks, training to near-zero training loss (with proper regularization) also improves validation performance.

Key Insight

Modern neural networks can be trained to **perfectly fit** (memorize) the training data while still **generalizing well** to new data. This requires:

- Very large networks (billions of parameters)
- Strong regularization (but not L2 alone)
- Proper training techniques

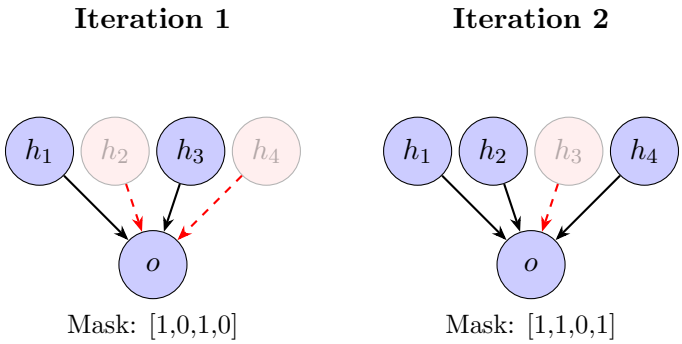
1.3 Dropout: Feature-Dependent Regularization

Definition 1.1 (Dropout)**.** Dropout is a regularization technique where, during training, we randomly set a fraction of neuron activations to zero.

1.3.1 Training Time

At each training iteration:

1. Sample a random binary mask $\mathbf{m} \in \{0,1\}^n$ where $P(m_i = 1) = p$
2. Apply element-wise multiplication: $\mathbf{h}' = \mathbf{m} \odot \mathbf{h}$
3. Continue forward pass with dropped-out activations



1.3.2 Test Time

At test time:

- **No dropout** — use all neurons
- **Scale weights** by dropout probability p to compensate

$$\mathbf{h}_{\text{test}} = p \cdot \mathbf{h} \tag{2}$$

Modern Implementation Note: In frameworks like PyTorch, the scaling is done during training (inverted dropout) rather than at test time.

1.3.3 Why Dropout Works

Multiple Interpretations:

1. **Prevents Feature Co-adaptation**
   - Network cannot rely on specific combinations of features
   - Must learn robust, redundant representations
   - Forces use of all available features
2. **Model Ensemble**
   - Training with dropout = training 2^n different subnetworks
   - Each mask creates a different network architecture
   - Test time approximates averaging all these networks

3. **Feature-Dependent Regularization**
   - Middle ground between Naive Bayes (features independent) and Logistic Regression (features fully dependent)
   - Weights learned in context of random subsets of other features

Practical Note

Dropout rate p typically ranges from 0.2 to 0.5. Common default: $p = 0.5$ for hidden layers.

1.4 Vectorization

Golden Rule

Never use for-loops in deep learning code (except for data loading)!
Always use vector/matrix/tensor operations.

Speed Comparison:

- Python for-loops: Baseline (1x)
- NumPy vectorized (CPU): 10-100x faster
- GPU vectorized: 100-1000x faster

Example 1.1 (Dropout Implementation)**. Bad (for-loop):**

```
for i in range(len(h)):
    if random() > 0.5:
        h[i] = 0
```

Good (vectorized):

```
mask = (torch.rand(h.shape) > 0.5).float()
h = h * mask
```

1.5 Parameter Initialization

Why Random Initialization?

Symmetry Breaking: If all weights start at the same value, all neurons compute the same function and receive the same gradients. The network effectively has only one feature, repeated many times.

1.5.1 Standard Approaches

1. **Small Random Values**

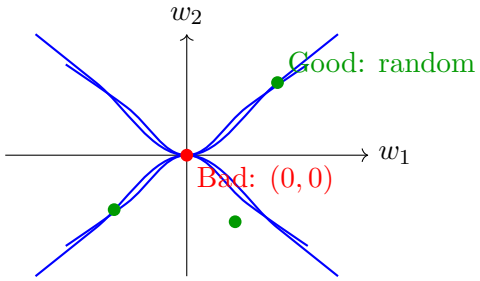
$$W_{ij} \sim \mathcal{U}(-\epsilon, \epsilon), \quad \epsilon \text{ small (e.g., 0.01)} \tag{3}$$

2. **Xavier/Glorot Initialization**

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right) \tag{4}$$

3. **He Initialization** (for ReLU networks)

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right) \tag{5}$$



Symmetric saddle point
No clear gradient direction

Modern Note: With layer normalization (covered later), initialization becomes less critical, but random initialization remains essential.

1.6 Optimizers

1.6.1 Beyond SGD

Standard SGD update:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L} \tag{6}$$

Problems with SGD:

- Highly sensitive to learning rate α
- Same learning rate for all parameters
- No adaptation to gradient history

1.6.2 Adaptive Optimizers

Core Idea: Maintain per-parameter learning rates based on gradient history.

AdaGrad (Duchi et al.)

$$g_t = \nabla_\theta \mathcal{L}_t \tag{7}$$

$$r_t = r_{t-1} + g_t^2 \quad \text{(accumulate squared gradients)} \tag{8}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{r_t + \epsilon}} \odot g_t \tag{9}$$

Issue: Learning rate monotonically decreases (can stall early).

Adam (Adaptive Moment Estimation)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \text{(first moment)} \tag{10}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \text{(second moment)} \tag{11}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(bias correction)} \tag{12}$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{13}$$

Default parameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Practical Recommendation

Start with Adam — it's robust and works well across many problems with default hyperparameters. For word embeddings (sparse updates), consider AdamW or specialized variants.

Other Variants

- **AdamW:** Adam with decoupled weight decay (better for embeddings)
- **RMSprop:** Similar to Adam but simpler (no momentum)
- **With momentum/Nesterov:** Additional acceleration techniques

2 Language Models

2.1 What is a Language Model?

Definition 2.1 (Language Model)**.** A language model is a system that assigns probabilities to sequences of words. Equivalently, it predicts a probability distribution over the next word given previous words.

Formal Definition: Given a context (sequence of words) $w_1, w_2, \ldots, w_{t-1}$, a language model computes:

$$P(w_t \mid w_1, \ldots, w_{t-1}) \tag{14}$$

such that $\sum_{w \in V} P(w \mid w_1, \ldots, w_{t-1}) = 1$ where V is the vocabulary.

Example 2.1 (Language Model in Action)**.** Context: *"The students opened their"*

Possible continuations with probabilities:

- $P(\text{books} \mid \text{context}) = 0.35$
- $P(\text{laptops} \mid \text{context}) = 0.25$
- $P(\text{notebooks} \mid \text{context}) = 0.20$
- $P(\text{bags} \mid \text{context}) = 0.10$
- $P(\text{other words} \mid \text{context}) = 0.10$

2.2 Probability of a Sequence

Using the **chain rule of probability**:

$$P(w_1, w_2, \ldots, w_n) = P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2) \cdots P(w_n \mid w_1, \ldots, w_{n-1}) \tag{15}$$

$$= \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1}) \tag{16}$$

Key Insight

A language model that can predict $P(w_t \mid w_{<t})$ can assign probabilities to **any** text sequence using the chain rule.

2.3 Applications of Language Models

Language models have been central to NLP since the 1980s:

1. **Text Generation**
   - ChatGPT, GPT-4, Claude
   - Autocomplete on smartphones
   - Google Search suggestions
2. **Speech Recognition**
   - Disambiguate acoustically similar words
   - "recognize speech" vs. "wreck a nice beach"
3. **Machine Translation**
   - Ensure fluent output in target language
4. **Spelling/Grammar Correction**
   - Identify and correct unlikely word sequences

3 N-gram Language Models

3.1 Historical Context

Timeline

- **1951:** Claude Shannon introduces the concept
- **1975-2012:** N-gram models dominate NLP
- **2012-present:** Neural language models take over

3.2 The N-gram Approach

Definition 3.1 (N-gram)**.** An n-gram is a contiguous sequence of n words from a text.

- Unigram ($n = 1$): single words
- Bigram ($n = 2$): pairs of words
- Trigram ($n = 3$): triples of words
- 4-gram, 5-gram, etc.

Naming Note: The proper Greek would be "monogram," "diagram," etc., but "unigram," "bigram" are standard in practice (following Claude Shannon's 1951 usage).

3.3 The Markov Assumption

Core Idea: Approximate the probability using only the most recent $n - 1$ words.

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) \tag{17}$$

Example 3.1 (Trigram Model ($n = 3$))**.**

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-2}, w_{t-1}) \tag{18}$$

Uses only the 2 previous words.

3.4 Estimating N-gram Probabilities

Use **maximum likelihood estimation** from corpus counts:

$$P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{\text{count}(w_{t-n+1}, \ldots, w_{t-1}, w_t)}{\text{count}(w_{t-n+1}, \ldots, w_{t-1})} \tag{19}$$

Example 3.2 (4-gram Probability Estimation)**.** Text: *"As the proctor started the clock, the students opened their ___"*

For 4-gram model, use context: *"students opened their"*

$$\begin{aligned} P(\text{books} \mid \text{students opened their}) &= \frac{\text{count(students opened their books)}}{\text{count(students opened their)}} \\ &= \frac{400}{1000} = 0.4 \end{aligned}$$

$$\begin{aligned} P(\text{exams} \mid \text{students opened their}) &= \frac{\text{count(students opened their exams)}}{\text{count(students opened their)}} \\ &= \frac{100}{1000} = 0.1 \end{aligned}$$

3.5 Problems with N-gram Models

3.5.1 Problem 1: Sparsity

Exponential growth!

Bigrams: $|V|^2 = 2.5 \times 10^9$
Trigrams: $|V|^3 = 1.25 \times 10^{14}$
4-grams: $|V|^4 = 6.25 \times 10^{18}$

Vocabulary: $|V| = 50,000$ words

Consequences:

- Most n-grams never observed in training data
- Zero probabilities for unseen n-grams
- Storage requirements explode with n

3.5.2 Problem 2: Zero Probabilities

Many perfectly valid word sequences will have zero probability because they weren't observed in training.

Example 3.3. Training corpus might contain:

- "students opened their books"
- "students opened their laptops"
- "students opened their exams"

But not:

- "students opened their accounts" $\Rightarrow P = 0$
- "students opened their frogs" $\Rightarrow P = 0$ (biology class!)

Why zeros are bad: $P(w_1, \ldots, w_n) = \prod_i P(w_i \mid \cdots)$ — one zero makes entire sequence probability zero!

3.6 Solutions to Sparsity

3.6.1 Smoothing (Add-δ)

Add a small count to all n-grams:

$$P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{\text{count}(w_{t-n+1}, \ldots, w_t) + \delta}{\text{count}(w_{t-n+1}, \ldots, w_{t-1}) + \delta|V|} \tag{20}$$

Typical: $\delta = 0.25$ or $\delta = 1$ (Laplace smoothing)

3.6.2 Backoff

If n-gram not observed, use shorter context:

$$P(w_t \mid w_{t-2}, w_{t-1}) = \begin{cases} \text{trigram estimate} & \text{if trigram seen} \\ \alpha \cdot P(w_t \mid w_{t-1}) & \text{if only bigram seen} \\ \beta \cdot P(w_t) & \text{if only unigram seen} \end{cases} \tag{21}$$

where α, β are backoff weights.

3.7 Practical Limitations

Conflicting Pressures

- **Want larger n:** More context $\Rightarrow$ better predictions
- **Want smaller n:** Less sparsity, smaller storage

Practical limit: $n = 5$ or 6 (rarely higher)

Famous Resource: Google N-grams (2006)

- Trained on 1 trillion words from web
- Released counts for n-grams up to $n = 5$
- Storage: many terabytes

3.8 Text Generation with N-grams

Algorithm 1 Generate Text with N-gram Model

```
Initialize context = [w_1, w_2] (e.g., "Today the")
while not done do
    Get probability distribution: P(w | context)
    Sample next word w_next from distribution
    Append w_next to generated text
    Update context = last n − 1 words
end while
```

Example 3.4 (Trigram Generation)**. Step 1:** Context = "today the"

Word	Probability
price	0.20
government	0.15
company	0.10
market	0.08
⋮	⋮

Sample: "price" ⇒ "today the price"

Step 2: Context = "the price"

Word	Probability
of	0.35
is	0.18
for	0.12
⋮	⋮

Sample: "of" ⇒ "today the price of"

Continue...

3.8.1 Example Generated Text

Trigram Model Output (2M word corpus)

"Today the price of gold per ton while production of shoe lasts and shoe industry the bank intervened just after it considered and rejected an IMF demand to rebuild depleted European stocks September 3rd in primary 76 cents a share"

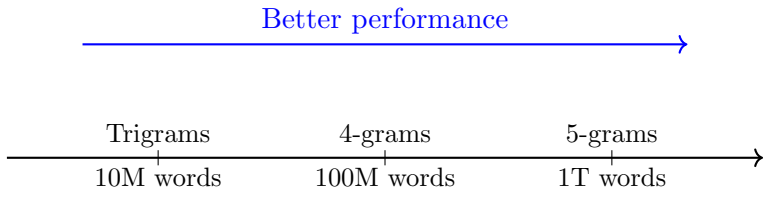
Observations:

- Mostly grammatical
- Local coherence (e.g., "the bank intervened just after it considered and rejected an IMF demand")
- No global coherence or meaning
- Random topic jumps

Improvements possible:

- More training data (10M, 100M, 1B words)
- Larger n (4-grams, 5-grams)
- Better smoothing techniques

3.9 The Scale Will Solve Everything Philosophy



Early 2010s philosophy:

More data + bigger n = better models

Historical Note

The same "scale solves everything" narrative that dominated n-gram language modeling (2000s-2010s) has returned with modern neural language models. But better model architectures matter too!

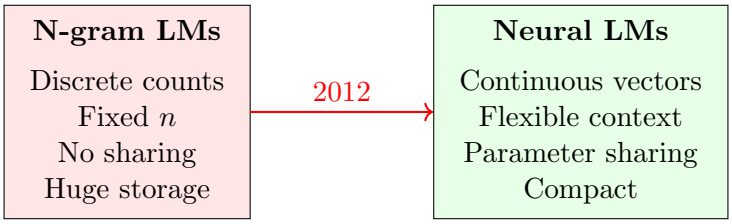
4 Transition to Neural Language Models

4.1 Limitations of N-gram Models

1. **Fixed context window:** Cannot use information beyond $n-1$ words
2. **No parameter sharing:** "students opened" and "pupils opened" treated as completely independent
3. **Storage explosion:** Need to store counts for every n-gram
4. **Sparsity:** Most possible n-grams never observed

4.2 What Neural LMs Offer

- **Distributed representations:** Words represented as dense vectors
- **Sharing:** Similar words have similar representations
- **Flexible context:** Can condition on arbitrary-length history
- **Compact storage:** Fixed number of parameters regardless of corpus size



4.3 Preview: Recurrent Neural Networks

Next lecture topics:

- Recurrent Neural Networks (RNNs) for language modeling
- Handling variable-length sequences
- Long Short-Term Memory (LSTM) networks
- Problems with RNNs (vanishing/exploding gradients)
- Modern alternatives: Transformers

5 Summary

Key Takeaways

1. **Modern Deep Learning Success** comes from:
   - Dropout and better regularization
   - Proper initialization and optimization (Adam)
   - GPU acceleration through vectorization
2. **Language Models** predict probability distributions over next words:

$$P(w_t \mid w_{<t})$$

3. **N-gram Models** (1950s-2012):
   - Simple, easy to build
   - Based on counting co-occurrences
   - Limited by sparsity and storage
   - Max practical size: 5-grams
4. **Neural LMs** overcome n-gram limitations through:
   - Distributed representations
   - Parameter sharing
   - Flexible-length context

5.1 Looking Ahead

Assignment 2: Implements dropout, Adam optimizer, and begins work with PyTorch
Assignment 3: Recurrent neural networks for language modeling
Future lectures:

- RNNs and LSTMs
- Attention mechanisms
- Transformer architecture
- Modern large language models

These notes cover CS224N Lecture 5: Language Models and Advanced Neural Network Concepts
For questions or corrections, please use the course Ed Discussion board.